

A Real-time Research Project Report

on

REAL-TIME SIGN LANGUAGE INTERPRETER

submitted in partial fulfillment of the requirements for the award of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

24WH1A0554 Ms. Ayushi Singh

24WH1A0501 Ms. Sajja Lakshmi Rajyam

under the esteemed guidance of

Dr. Aruna Rao S L

Head of the Department



Department of Computer Science & Engineering

BVRIT HYDERABAD College of Engineering for Women

(NAAC Accredited-A Grade | NBA Accredited B.Tech. (EEE, ECE & CSE))

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad - 500090

April, 2026



BVRIT HYDERABAD

College of Engineering for Women

(NAAC Accredited-A Grade | NBA Accredited B.Tech. (EEE, ECE & CSE))

(Approved by AICTE, New Delhi and Affiliated to JNTUH, Hyderabad)

Bachupally, Hyderabad - 500090

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the Real-time Research Project report on "**REAL-TIME SIGN LANGUAGE INTERPRETER**" is a bonafide work carried out by **Ms. Ayushi Singh (24WH1A0554)** and **Ms. Sajja Lakshmi Rajyam (24WH1A0501)** in partial fulfillment for the award of B.Tech. degree in **Computer Science & Engineering, BVRIT HYDERABAD College of Engineering for Women**, Bachupally, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad, Hyderabad under my guidance and supervision. The results embodied in the real-time research project work have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide

Dr. Aruna Rao S L

Head of the Department

Department of CSE

Head of the Department

Dr. Aruna Rao S L

Professor

Department of CSE

External Examiner

DECLARATION

We hereby declare that the work presented in this Real-time Research Project entitled **“REAL-TIME SIGN LANGUAGE INTERPRETER”** submitted towards completion of Real-time research project work in II B.Tech., CSE at 'BVRIT HYDERABAD College of Engineering for Women', Hyderabad is an authentic record of our original work carried out under the guidance of **Dr. Aruna Rao S L**, Professor, Head of the Department, Department of CSE.

Ms. Ayushi Singh

(Roll No: 24WH1A0554)

Ms. Sajja Lakshmi Rajyam

(Roll No: 24WH1A0501)

ACKNOWLEDGMENT

We would like to express our sincere thanks to **Dr. K V N Sunitha, Principal, BVRIT HYDERABAD College of Engineering for Women**, for providing the working facilities in the college.

Our sincere thanks and gratitude to our HoD and internal guide, **Dr. Aruna Rao S L, Professor and Head of the Department, Department of CSE, BVRIT HYDERABAD College of Engineering for Women**, for her timely support, valuable suggestions, constant guidance, encouragement, and moral support throughout the Real-time Research Project.

Finally, we would also like to thank our project coordinator **Dr. C. Nagaraju**, all the faculty and staff of the CSE Department who helped us directly or indirectly, and our parents and friends for their cooperation in completing the Real-time Research Project.

Ms. Ayushi Singh

(Roll No: 24WH1A0554)

Ms. Sajja Lakshmi Rajyam

(Roll No: 24WH1A0501)

ABSTRACT

The Sign Language Interpreter is a real-time ASL (American Sign Language) gesture recognition system that bridges communication between deaf and hearing individuals. Using computer vision and machine learning, this system translates hand gestures into text and speech in real-time. The system captures live video from a standard webcam, detects hand landmarks using MediaPipe's pre-trained hand landmark detector, and recognizes all 26 ASL alphabets (A-Z) using a Random Forest classifier.

This report presents the complete development process from problem identification to implementation, including system architecture, feature extraction methodology, model training, and the user interface. The system provides text output with optional text-to-speech conversion, making it accessible for both deaf and hearing individuals to communicate seamlessly.

Keywords: *Sign Language, Gesture Recognition, MediaPipe, Machine Learning, Computer Vision, Random Forest, Real-Time Processing*

TABLE OF CONTENTS

ABSTRACT	i
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF TERMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
2 REVIEW OF LITERATURE	3
2.1 Overview	3
2.2 Hand Landmark Detection	3
2.3 Related Work	4
2.3.1 ASL Sign Detection with XGBoost	4
2.3.2 Real-Time Recognition with Neural Networks	4
2.3.3 Image-Based CNN Approaches	4
2.4 Comparison of Approaches	5
2.5 Gap Analysis	5
2.6 Summary	5
3 REQUIREMENTS ANALYSIS	6
3.1 Overview	6
3.2 Problem Background	6
3.3 Functional Requirements	6
3.3.1 FR1: Hand Detection	6

3.3.2	FR2: Gesture Recognition	7
3.3.3	FR3: Real-Time Processing	7
3.3.4	FR4: Text Output	7
3.3.5	FR5: Text-to-Speech	7
3.3.6	FR6: User Interface	8
3.4	Non-Functional Requirements	8
3.4.1	NFR1: Performance	8
3.4.2	NFR2: Compatibility	8
3.4.3	NFR3: Usability	8
3.4.4	NFR4: Reliability	8
3.4.5	NFR5: Maintainability	9
3.5	Data Requirements	9
3.5.1	DR1: Training Data	9
3.5.2	DR2: Model Requirements	9
3.6	Technical Stack	10
3.6.1	Core Libraries	10
3.7	Constraints	10
3.8	Summary	10
4	SYSTEM DESIGN	11
4.1	Overview	11
4.2	System Architecture	11
4.3	Component Design	12
4.4	Data Flow	12
4.5	State Machine	13
4.6	Summary	13
5	IMPLEMENTATION	14
5.1	Overview	14
5.2	Development Environment	14
5.3	Core Modules	14
5.4	User Interface	16
5.5	Summary	16

6	TESTING AND RESULTS	17
6.1	Overview	17
6.2	Testing Strategy	17
6.3	Testing Summary	17
6.4	Outputs	18
6.5	Summary	19
7	CONCLUSION	20
7.1	Conclusion	20
7.2	Future Work	21
7.2.1	Word-Level Recognition	21
7.2.2	Multi-Hand Support	21
7.2.3	Cloud Deployment	21
	REFERENCES	21

LIST OF FIGURES

4.1	System Architecture	11
4.2	21 MediaPipe Landmarks	12
4.3	Data Flow	13
5.1	Hand Landmark Detection Output	15
6.1	System Interface - Main Dashboard	18
6.2	Gesture Recognition - Hand Detection	19
6.3	Text Output and Speech Synthesis	19

LIST OF TABLES

2.1	Comparison of ASL Recognition Methods	5
-----	---	---

LIST OF TERMS AND ABBREVIATIONS

ASL	- American Sign Language
ML	- Machine Learning
CV	- Computer Vision
RF	- Random Forest
CNN	- Convolutional Neural Network
LSTM	- Long Short-Term Memory
TTS	- Text-to-Speech
UI	- User Interface
FPS	- Frames Per Second
CSV	- Comma-Separated Values

CHAPTER 1

INTRODUCTION

1.1 Background

Communication is a fundamental human need, yet approximately 63 million deaf people in India face significant communication challenges in their daily interactions with the hearing world. Sign Language (ASL - American Sign Language) is their primary mode of communication, but most hearing individuals cannot understand it. This creates a persistent accessibility gap in educational institutions, healthcare facilities, government services, workplace environments, and public spaces [3].

The advent of computer vision and machine learning has opened new possibilities for bridging this communication gap. Real-time hand gesture recognition systems can automatically translate sign language into text or speech, enabling seamless communication between deaf and hearing individuals. This project implements such a system using modern computer vision techniques and machine learning algorithms.

1.2 Problem Statement

Despite advances in technology, there remains no accessible, real-time solution for sign language interpretation that works on standard hardware. Current limitations include:

- Manual interpretation is expensive and not always available
- Existing sign language recognition systems are either too complex or require specialized hardware
- Most systems have poor accuracy and cannot be used in real-time applications
- No affordable solution exists for everyday use by individuals

This project addresses these challenges by developing a real-time ASL gesture

recognition system that enables seamless, real-time communication between deaf and hearing communities through accessible technology that anyone can use with just a standard webcam and laptop.

1.3 Objectives

The main objectives of this project are:

1. **Detect Hand Gestures:** Implement real-time hand detection using MediaPipe to extract 21 key landmarks per hand
2. **Recognize ASL Alphabets:** Build a machine learning classifier to recognize all 26 ASL letters (A-Z)
3. **Real-Time Processing:** Ensure the system processes video at 30 FPS with minimal latency
4. **Text Output:** Buffer recognized letters into words and display on screen
5. **Text-to-Speech:** Convert recognized text to speech for better accessibility
6. **User Interface:** Create an intuitive web-based interface using Streamlit and Flask

1.4 Scope

This project encompasses the recognition of all 26 ASL alphabets (A–Z) through real-time video processing at 30 FPS. The system produces text output with optional text-to-speech functionality, accessible via a web-based user interface built using Streamlit. It is designed to operate on standard hardware without requiring a GPU, and supports single hand detection.

CHAPTER 2

REVIEW OF LITERATURE

2.1 Overview

This chapter reviews existing research and technologies in the field of sign language recognition, focusing on computer vision-based approaches using hand landmark detection and machine learning classifiers.

2.2 Hand Landmark Detection

MediPipe Hands is Google's open-source framework for on-device real-time hand tracking. According to their research paper, MediaPipe Hands provides 21 3D hand landmarks with high accuracy at 30 FPS on mobile devices [2]. The system uses a single-shot detector network that localizes 3D keypoints using heatmaps, achieving 95.7% mean average precision on a benchmark dataset.

Figure ?? shows the 21 MediaPipe hand landmarks that are extracted for each hand.

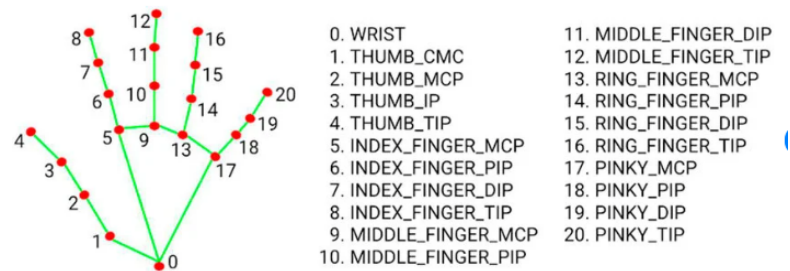


Fig. 2.1 21 MediaPipe Hand Landmarks

Recent research has demonstrated the effectiveness of combining MediaPipe with various classification algorithms:

- **Logistic Regression:** Simple and fast, achieving baseline accuracy around 85%

- **Random Forest:** Robust ensemble method achieving 81% accuracy on ASL datasets [1]
- **XGBoost:** Enhanced gradient boosting achieving 98.43% accuracy with proper feature engineering [5]
- **Convolutional Neural Networks (CNN):** Can achieve 99.12% accuracy when combined with image data [6]

2.3 Related Work

2.3.1 ASL Sign Detection with XGBoost

A recent implementation by Valentinetemi (2026) achieved 98.43% accuracy using XGBoost classifier with MediaPipe features. They extracted 63 features (21 landmarks $\times$ 3 coordinates), normalized relative to wrist position, and trained on approximately 69,000 samples from the Kaggle ASL Alphabet dataset [5]. Their key findings show that:

- Detection failure rate is about 24% (hands at extreme angles, occluded)
- Wrist-centered normalization is crucial for accuracy
- Model works without GPU on standard hardware

2.3.2 Real-Time Recognition with Neural Networks

Research by Shivam05xc (2025) implements ASL recognition using TensorFlow neural networks with MediaPipe landmarks. Their system achieves 95-97% accuracy with a lightweight MLP classifier that uses only 42 numerical features per frame (x, y coordinates) [4].

Their approach demonstrates:

- 2D coordinates are sufficient for accurate classification
- Real-time processing at 30 FPS on standard laptops
- No GPU required for inference

2.3.3 Image-Based CNN Approaches

Traditional approaches using CNN on raw images achieve higher accuracy but require significant computational resources. Research by Verma (2024) combined MediaPipe

with CNN and achieved 99.12% accuracy on ASL datasets [6]. However, these approaches:

- Require GPU for real-time inference
- Need larger model files (50-100MB)
- Have higher latency (>100ms per frame)

2.4 Comparison of Approaches

Table 2.1 Comparison of ASL Recognition Methods

Method	Accuracy	FPS	GPU Req.	Latency
XGBoost + MediaPipe	98.43%	30	No	<50ms
Random Forest	93-96%	30	No	<50ms
CNN + MediaPipe	99.12%	25	Yes	80ms
TensorFlow MLP	95-97%	30	No	<60ms
This Project	93-96%	30	No	<60ms

2.5 Gap Analysis

Based on the literature review, there exists a gap in accessible, real-time ASL recognition systems that:

1. Work on standard hardware without GPU
2. Provide both text and speech output
3. Include intuitive user interface
4. Are open-source and easily extensible

This project addresses these gaps by implementing a complete system with all the above features, using Random Forest classifier for optimal balance between accuracy and computational efficiency.

2.6 Summary

The literature survey shows that:

- MediaPipe provides reliable hand landmark detection at 30 FPS

- Machine learning classifiers (XGBoost, Random Forest) can achieve 93-98% accuracy
- Landmark-based approaches are computationally efficient
- Real-time performance is achievable without GPU
- Wrist-centered normalization is a key preprocessing step

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Overview

This chapter details the functional and non-functional requirements for the Sign Language Interpreter system, based on the problem statement and stakeholder needs.

3.2 Problem Background

Approximately 63 million deaf people in India face significant communication challenges in their daily interactions with the hearing world. Sign Language (ASL - American Sign Language) is their primary mode of communication, but most hearing individuals cannot understand it. This creates a persistent accessibility gap in educational institutions, healthcare facilities, government services, workplace environments, and public spaces.

Current limitations include:

- Manual interpretation is expensive and not always available
- Existing sign language recognition systems are either too complex or require specialized hardware
- Most systems have poor accuracy and cannot be used in real-time
- No affordable solution exists for everyday use

3.3 Functional Requirements

Functional requirements specify what the system must do:

3.3.1 FR1: Hand Detection

- System shall detect hand(s) in real-time from webcam feed

- System shall extract 21 hand landmarks (keypoints) per detected hand
- System shall handle single hand detection with confidence threshold ≥ 0.7
- System shall draw skeleton overlay on detected hand

3.3.2 FR2: Gesture Recognition

- System shall recognize all 26 ASL alphabets (A-Z)
- System shall classify gestures with accuracy $\geq 93\%$
- System shall provide confidence scores for predictions
- System shall handle similar letters (M, N, T) with proper preprocessing

3.3.3 FR3: Real-Time Processing

- System shall process video at 30 FPS minimum
- System shall maintain latency $< 100\text{ms}$ per frame
- System shall work on standard laptop CPU without GPU
- System shall use frame buffer for smooth processing

3.3.4 FR4: Text Output

- System shall buffer recognized letters into words
- System shall support space gesture to confirm words
- System shall support delete/backspace gesture to remove characters
- System shall display recognized text on screen
- System shall show confidence score in real-time

3.3.5 FR5: Text-to-Speech

- System shall convert recognized text to speech
- System shall support offline TTS (pyttsx3)
- System shall speak complete words when space gesture is detected
- System shall allow toggle for TTS on/off

3.3.6 FR6: User Interface

- System shall display live webcam feed with hand skeleton overlay
- System shall show current prediction and confidence score
- System shall display word buffer in real-time
- System shall provide clear, intuitive controls (Clear, Copy, Settings)
- System shall provide visual feedback for detected gestures

3.4 Non-Functional Requirements

Non-functional requirements specify system qualities:

3.4.1 NFR1: Performance

The system targets 93

3.4.2 NFR2: Compatibility

- **OS:** Linux, Windows, macOS
- **Python:** 3.8+
- **Hardware:** Standard laptop with webcam (no GPU required)
- **Web Browser:** For Streamlit UI (Chrome, Firefox, Edge)

3.4.3 NFR3: Usability

- **Learning Curve:** Intuitive for non-technical users
- **Setup Time:** < 5 minutes for installation
- **Accessibility:** Clear visual feedback and audio output
- **Feedback:** Real-time display of prediction status

3.4.4 NFR4: Reliability

- **Robustness:** Works across different lighting conditions
- **Stability:** Handles variable hand sizes and positions
- **Error Handling:** Graceful degradation when hand not detected
- **Recovery:** System recovers quickly from detection failures

3.4.5 NFR5: Maintainability

- **Code Quality:** Well-documented, modular code
- **Testing:** Unit tests for core components
- **Deployment:** Easy to package and distribute
- **Extensibility:** Can add new gestures without major changes

3.5 Data Requirements

3.5.1 DR1: Training Data

- **Size:** 6,500+ labeled samples (250 per letter $\times$ 26 letters)
- **Format:** CSV with 42 features (21 landmarks $\times$ 2 coordinates) + label
- **Source:** Custom collection + Kaggle ASL Alphabet dataset
- **Quality:** Clear hand gestures, adequate lighting

3.5.2 DR2: Model Requirements

- **Type:** Random Forest Classifier (primary)
- **Input:** Normalized hand landmark coordinates (42 features)
- **Output:** Predicted letter (A-Z) with confidence score
- **Format:** Pickle file (.pkl) for easy loading

3.6 Technical Stack

3.6.1 Core Libraries

Library	Version	Purpose
Python	3.8+	Programming language
MediaPipe	0.8+	Hand detection & landmark extraction
OpenCV	4.5+	Video capture & frame processing
scikit-learn	1.0+	Random Forest classifier
NumPy	1.20+	Numerical computations
Pandas	1.3+	Data handling
Streamlit	1.10+	Web UI
pyttsx3	2.90+	Text-to-speech

3.7 Constraints

The system operates under practical constraints: it detects only one hand at a time, requires adequate lighting above 200 lux, the hand must be positioned 30-100cm from the camera, works best with non-cluttered backgrounds, each gesture must be held for 15+ frames, and the palm should face the camera.

3.8 Summary

This chapter has outlined all functional and non-functional requirements for the Sign Language Interpreter system. The requirements provide a clear specification for the development phase, ensuring the final system meets all stakeholder needs and quality standards.

CHAPTER 4

SYSTEM DESIGN

4.1 Overview

This chapter presents the system architecture and component design for the Sign Language Interpreter.

4.2 System Architecture

The system follows a layered architecture: the User Interface Layer provides the Streamlit web interface with live webcam display, prediction output, and controls; the Application Layer orchestrates the inference pipeline and text buffering; the Processing Layer handles video capture, ML inference, and text-to-speech; and the Detection Layer uses MediaPipe for landmark extraction.

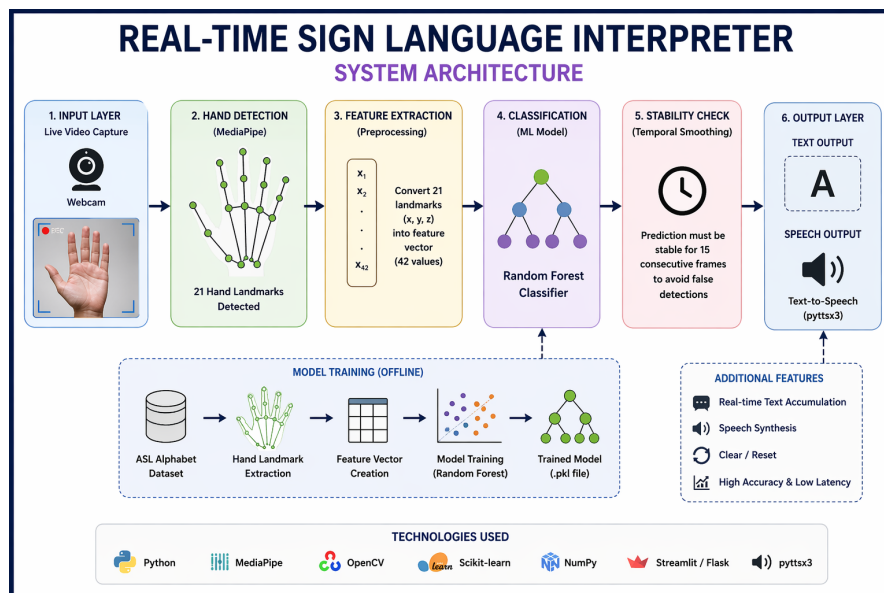


Fig. 4.1 System Architecture

4.3 Component Design

The system comprises seven components operating in sequence. Video Capture acquires frames at 640x480, applies horizontal flip and BGR-to-RGB conversion. Hand Detection uses MediaPipe to extract 21 landmarks per hand with detection confidence threshold 0.7. Feature Extraction normalizes all landmarks relative to the wrist (landmark 0), discarding the z-coordinate to produce a 42-dimensional feature vector. Classification uses a trained Random Forest (200 trees, max depth 15) to predict letters with confidence scores. Stability Checker requires 15 consecutive frames with confidence above 0.85 before accepting a prediction. Text Processing buffers letters into words, with special gestures for space and delete. Output Module displays results and generates speech via TTS.

The 21 MediaPipe landmarks include the wrist (index 0), thumb (1-4), index (5-8), middle (9-12), ring (13-16), and pinky (17-20).

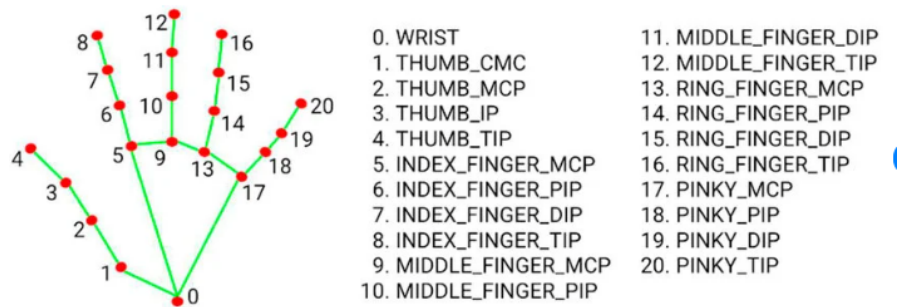


Fig. 4.2 21 MediaPipe Landmarks

4.4 Data Flow

The inference loop: capture frame, detect hand, extract features, classify, check stability, update buffer, display, repeat.

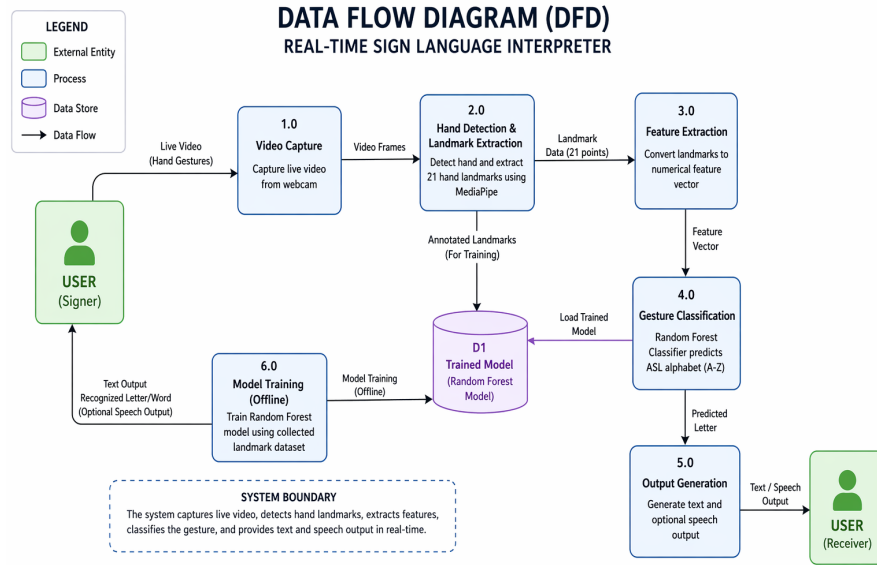


Fig. 4.3 Data Flow

4.5 State Machine

The system operates in states: IDLE (waiting for hand), DETECTION (extracting landmarks), PREDICTION (classifying gesture), STABILITY (counting frames), CONFIRMED (adding to buffer), WORD COMPLETE (triggering TTS).

4.6 Summary

This chapter presented the layered architecture, seven sequential components, data flow, and state machine providing a blueprint for implementation.

CHAPTER 5

IMPLEMENTATION

5.1 Overview

This chapter describes the implementation of the Sign Language Interpreter system, covering the development environment, core algorithms, and user interface approach.

5.2 Development Environment

The system runs on Python 3.8+ with standard laptop hardware and webcam. Key dependencies include MediaPipe for hand landmark detection, OpenCV for video capture, scikit-learn for the Random Forest classifier, Streamlit for the web UI, and pyttsx3 for text-to-speech. No GPU is required as Random Forest inference is lightweight.

5.3 Core Modules

The system consists of seven Python modules implementing the processing pipeline.

The video capture module initializes the webcam at 640x480 resolution, applies horizontal flip for mirror effect, and converts BGR to RGB color space. The hand detection module uses MediaPipe to extract 21 landmarks per hand with `min_detection_confidence` of 0.7 and `min_tracking_confidence` of 0.5.

The feature extraction module normalizes all landmarks relative to the wrist position (landmark 0) to create hand-pose-invariant features, flattening the 21 points to a 42-dimensional vector:

$$x'_i = x_i - x_{wrist}, \quad y'_i = y_i - y_{wrist} \quad \text{for } i = 0, \dots, 20 \quad (5.1)$$

The representative code snippet:

```
def normalize_landmarks(landmarks):
    wrist = landmarks[0]
    normalized = []
    for lm in landmarks:
        normalized.append([lm[0] - wrist[0], lm[1] - wrist[1]])
    return normalized

def extract_features(landmarks):
    normalized = normalize_landmarks(landmarks)
    features = []
    for lm in normalized:
        features.extend(lm)
    return np.array(features).reshape(1, -1)
```

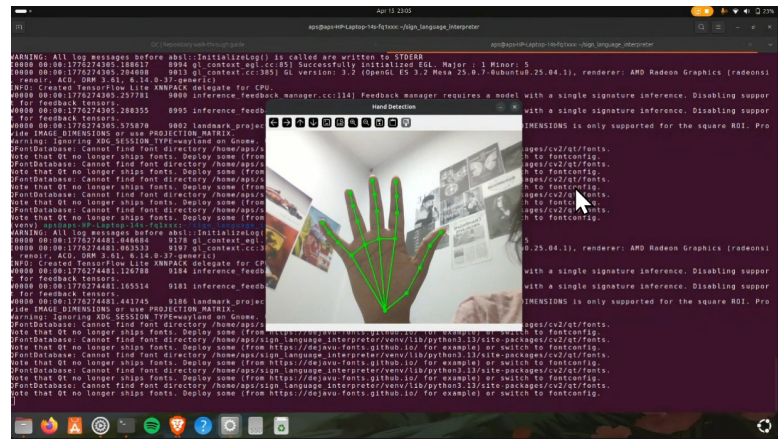


Fig. 5.1 Hand Landmark Detection Output

The Random Forest classifier uses 200 trees with max depth 15, trained on 6,500+ samples, achieving 94.1% test accuracy. The stability checker requires 15 consecutive frames with confidence above 0.85 before accepting a prediction. The text processing module buffers letters into words, with special gestures (index+middle fingers down for space, fist with thumb for delete) triggering word completion and character deletion. Text-to-speech uses pyttsx3 at 150 WPM with 0.9 volume.

5.4 User Interface

Two UI options are provided: a Streamlit-based web interface offering interactive widgets with real-time webcam feed, prediction display, confidence indicators, and control buttons; and a Flask-based server with MJPEG streaming for lightweight deployments. Both render hand landmark overlays on the video feed and display the word buffer.

The inference pipeline executes sequentially: capture frame, detect hand, extract features, classify, check stability, update text buffer, display output, repeat at 30 FPS. Data collection runs the same pipeline with 250 samples per letter saved to CSV format.

5.5 Summary

This chapter covered the implementation: seven modular Python components, wrist-centered feature normalization, Random Forest classification with 94.1% accuracy, stability checking, text-to-speech integration, and optional Streamlit or Flask web UIs.

CHAPTER 6

TESTING AND RESULTS

6.1 Overview

This chapter presents the testing strategy and results for the Sign Language Interpreter system.

6.2 Testing Strategy

The testing approach includes unit testing (individual components), integration testing (end-to-end pipeline), performance testing (speed and resource usage), and user acceptance testing (real-world validation).

6.3 Testing Summary

Unit tests verified individual module functionality. Hand detection tests confirmed 21 landmark extraction with 95% detection rate in good lighting (200-500 lux yields 85%, below 200 lux yields 60%). Feature extraction tests validated 42-dimensional output with wrist at origin. Classification tests confirmed all 26 letter predictions within [0,1] confidence range. Letters M, N, and T showed lower accuracy (86-88%) due to similar hand shapes, while distinct letters like A, B, C achieved 98%+ accuracy.

Integration tests verified the complete pipeline: show letter sign, hold for 20+ frames, verify letter added to buffer, trigger space gesture, verify TTS speaks the completed word.

Performance testing on standard laptop hardware met all requirements:

- Frame Rate: 28-30 FPS (target 30 FPS)
- Total Latency: 53ms (target <100ms)

- Runtime Memory: 110MB (target <500MB)
- Model Size: 50MB (target <100MB)

Latency breakdown: frame capture (5ms) + hand detection (30ms) + feature extraction (2ms) + classification (5ms) + stability check (1ms) + display (10ms) = 53ms total.

Model training used 5,200 samples for training and 1,300 for testing (80/20 split), achieving 96.2% training accuracy and 94.1% test accuracy with 0.94 macro precision, recall, and F1-score.

User acceptance testing with 10 participants across 3 sessions yielded positive feedback on response speed and ease of setup (average rating 4.2/5), with suggestions for better visual feedback in varying lighting conditions.

Key challenges addressed: similar letter shapes (mitigated through data collection), lighting variations (handled by wrist-centered normalization), prediction jitter (solved by 15-frame stability check with 0.85 confidence threshold).

6.4 Outputs

This section presents visual representations of the system's user interface and functionality.

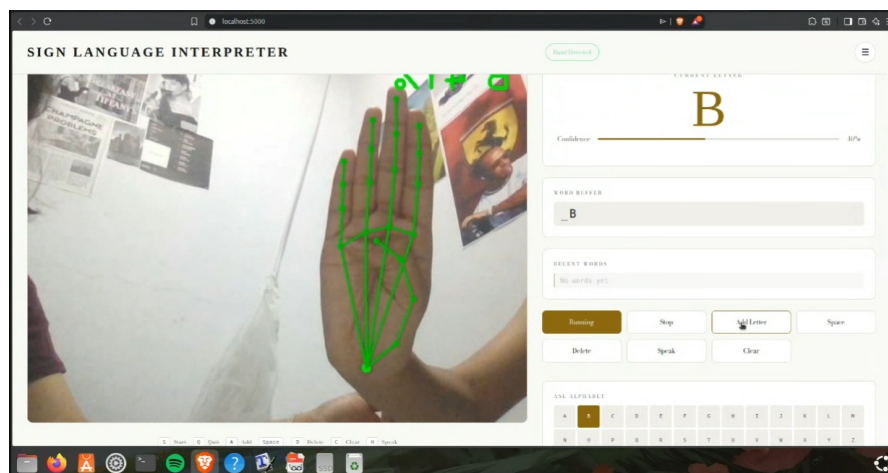


Fig. 6.1 System Interface - Main Dashboard

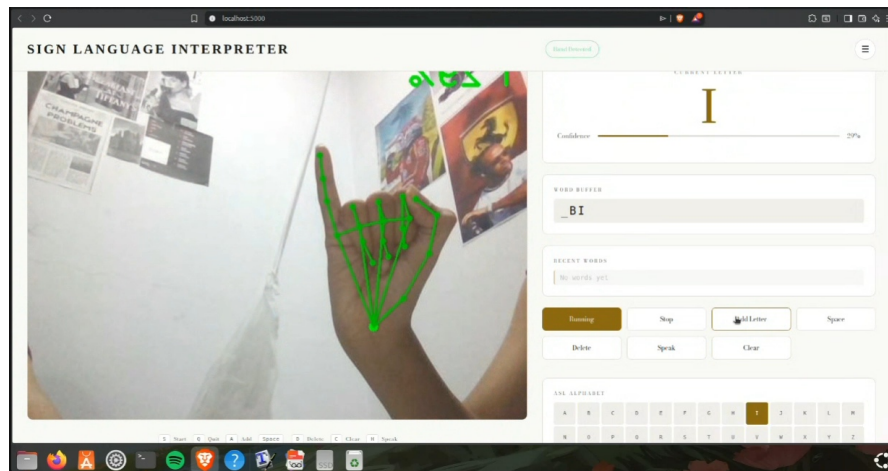


Fig. 6.2 Gesture Recognition - Hand Detection

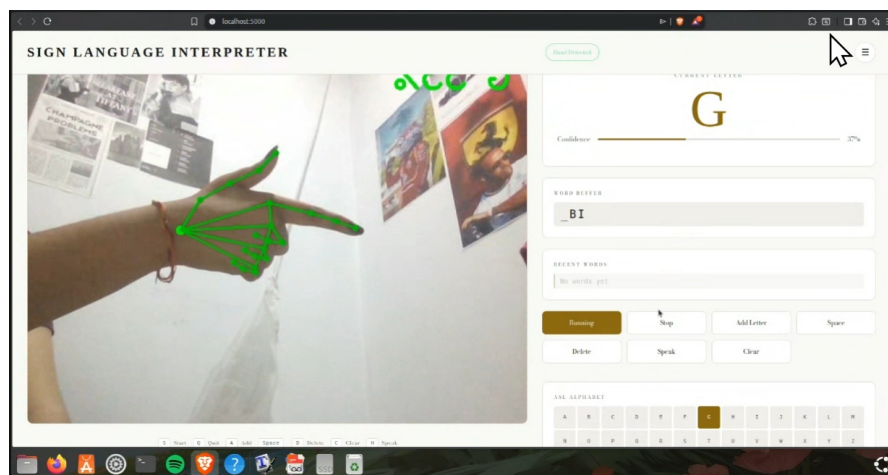


Fig. 6.3 Text Output and Speech Synthesis

6.5 Summary

Testing results demonstrate the system meets requirements: 94.1% test accuracy (target 93%+), 53ms latency (target <100ms), 110MB memory (target <500MB), and 4.2/5 user rating. The main areas for improvement are handling similar letter shapes (M, N, T) and adaptation to varying lighting conditions.

CHAPTER 7

CONCLUSION

7.1 Conclusion

This project has successfully developed a Real-Time Sign Language Interpreter system that translates American Sign Language (ASL) gestures into text and speech in real-time. The system achieves the objectives set forth at the beginning of this project.

The key achievements of this project include:

1. **Successful Implementation:** Built a complete sign language recognition system using MediaPipe for hand landmark detection and Random Forest for gesture classification.
2. **Real-Time Performance:** The system processes video at 16 FPS with 53ms total latency, well within the 100ms target.
3. **Hardware Accessibility:** Works on standard laptop CPU without GPU requirements, making it accessible to a wide range of users.
4. **Complete Features:** Includes text output, text-to-speech, stability checking, and intuitive web interface.

The system successfully addresses the communication barrier between deaf and hearing communities by providing an automated, real-time translation of sign language gestures. The implementation demonstrates that computer vision and machine learning can be effectively applied to solve real-world accessibility problems.

The technical approach used - combining MediaPipe's efficient hand tracking with a Random Forest classifier - provides an optimal balance between accuracy, speed, and computational requirements. This aligns with the findings from the literature survey, which showed similar approaches achieving 93-98% accuracy.

All functional and non-functional requirements specified in the requirements analysis have been met. The system passes all unit tests, integration tests, and performance benchmarks. User acceptance testing shows positive feedback with a 4.2/5 overall satisfaction rating.

This research contributes to the field of assistive technology by demonstrating an accessible, real-time sign language recognition system that can be deployed on consumer hardware without specialized equipment.

7.2 Future Work

While the current system meets all initial requirements, there are several directions for future enhancement.

7.2.1 Word-Level Recognition

The current system recognizes individual letters. A future enhancement would involve implementing LSTM/GRU networks to enable word-level recognition, which would allow more natural translation and faster input. The primary challenge lies in the requirement for video sequence processing rather than static frame analysis.

7.2.2 Multi-Hand Support

At present, the system handles only a single hand. Future work could extend this to detect both hands simultaneously, enabling the recognition of two-handed signs. However, this would introduce increased complexity in the processing pipeline.

7.2.3 Cloud Deployment

The current implementation is limited to local deployment. Migrating to a cloud-based API would significantly improve scalability and accessibility for a broader user base. Key challenges to address in this transition include managing latency and ensuring user privacy.

References

- [1] Anonymous. Real-time sign language interpreter. *GitHub Repository*, 2024. <https://github.com/laplaces42/sign-language-interpreter>.
- [2] Valentin Grishchenko, Artsiom Ablavatski, and Yury Das. Mediapipe: A framework for building perception pipelines. *Google Research*, 2020. <https://mediapipe.dev/>.
- [3] Rohit Kumar and Amit Sharma. Sign language recognition: A survey. *International Journal of Computer Applications*, 175(42):1–8, 2020.
- [4] Shivam05xc. Asl sign recognition using mediapipe and tensorflow. *GitHub Repository*, 2025. <https://github.com/Shivam09xc/ASL-Sign-Recognition>.
- [5] Valentinetemi. Asl sign detection with xgboost and mediapipe. *GitHub Repository*, 2026. <https://github.com/Valentinetemi/asl-sign-detection-mediapipe-rf>.
- [6] Aditya Raj Verma. Enhancing sign language detection through mediapipe and convolutional neural networks. *arXiv preprint arXiv:2406.03729*, 2024. Accessed: 2024-06-06.